

Rapport Projet Analyse Syntaxique L3 2025

Ben Hamouda Maissa
Gavau--Pelissier Liam

Sommaire :

1. Introduction
2. Interface utilisateur
3. Architecture
4. Choix d'implémentation
5. Problèmes rencontrés

Introduction

L'objectif de ce projet est la conception et l'implémentation d'un analyseur lexical et syntaxique pour le langage TPC, un sous-ensemble du langage C.

Pour mener à bien cette tâche, nous avons utilisé les générateurs d'analyseurs flex et bison, conformément aux contraintes imposées.

Le cœur du travail a consisté non seulement à reconnaître la grammaire de base du langage, mais également à l'étendre pour intégrer la gestion des types structures (déclarations, champs, types imbriqués), tout en respectant une syntaxe proche du C mais simplifiée.

L'analyseur produit a pour vocation de vérifier la validité lexicale et syntaxique du code source et, en l'absence d'erreurs, de construire et d'afficher l'arbre abstrait correspondant sur la sortie standard

Interface utilisateur

Interface utilisateur

L'analyseur tpcas accepte les options suivantes :

- -t, --tree : affiche l'arbre syntaxique abstrait
- -h, --help : affiche l'aide

Le programme doit être compilé avec la fonction make exécuté dans un terminal depuis le répertoire du projet

Puis le programme peut être lancé avec ./tpcas [OPTIONS] < fichier.tpc ou ./tpcas [OPTIONS] fichier.tpc.

Les valeurs de retour respectent la spécification :

- **0** : aucune erreur
- **1** : erreur lexicale ou syntaxique
- **2+** : erreur de ligne de commande ou autre

Architecture

Le projet est organisé selon la structure demandée :

ProjetASL3_BenHamouda_Gavau-Pelissier/

```
|— src/      # Fichiers sources (lex_proj.lex, bison_proj.y, tree.c, tree.h)
|— bin/      # Exécutable tpcas
|— obj/      # Fichiers objets intermédiaires
|— rep/      # Rapport du projet
└— test/     # Jeux de tests
    |— good/  # Programmes corrects
    └— syn-err/ # Programmes avec erreurs
```

Composants principaux

- **lex_proj.lex** : Analyseur lexical généré avec flex, responsable de la reconnaissance des lexèmes (mots-clés, identificateurs, opérateurs, commentaires, etc.)
- **bison_proj.y** : Analyseur syntaxique généré avec bison, qui vérifie la conformité du programme à la grammaire TPC et construit l'arbre abstrait
- **tree.c / tree.h** : Module de gestion de l'arbre syntaxique abstrait, permettant la création, manipulation et affichage de l'arbre

Choix d'implémentation

I. Analyseur lexical (Flex)

Pour le fichier .lex, nous nous sommes basés sur le travail effectué pendant les TP. Les principales modifications concernent :

- **Gestion des commentaires** : Nous avons implémenté un état exclusif <COMMENT> pour gérer les commentaires multi-lignes /* ... */ et les commentaires de ligne //. Cette approche permet d'ignorer correctement le contenu des commentaires tout en incrémentant le compteur de lignes pour le print d'erreur.
- **Reconnaissance du mot-clé struct** : Ajout du token STRUCT pour permettre la déclaration et l'utilisation des structures.
- **Point d'accès aux champs** : Le caractère . est reconnu comme un token pour permettre l'accès aux champs des structures (struct.field).
- **Gestion des caractères littéraux** : Nous avons implémenté la reconnaissance des caractères échappés (\n, \t, \') conformément à la spécification.

II. Analyseur syntaxique (Bison)

Nous sommes partis du fichier fourni dans le sujet et l'avons étendu progressivement en ajoutant plusieurs règle dont :

II.1 Déclarations globales et locales

Une règle DeclGlobales qui permet de regrouper à la fois :

- Les déclarations de structures : `STRUCT IDENT '{' Champs '}' ';' ;`
- Les déclarations de variables de types simples : `TYPE Declarateurs ';' ;`
- Les déclarations de variables de types structures : `STRUCT IDENT Declarateurs ';' ;`

Cette fusion était nécessaire pour éviter les conflits shift/reduce entre déclarations de structures et déclarations de variables de type structure le plus grand problème qu'on a rencontré et qu'on explique un peu plus bas dans le rapport.

De même, DeclLocales utilise la même logique pour les déclarations à l'intérieur des fonctions.

II.2 Affectations de champs

L'accès aux champs s'effectue via la syntaxe `variable.champ1.champ2....`. Nous avons créé :

- Une règle Var récursive qui capture les accès en chaîne : `.IDENT Var`
- Un nœud `L_FIELD_ACCESS` dans l'arbre abstrait qui regroupe tous les identificateurs de la chaîne d'accès

II.3 Fonctions avec types structures

Les en-têtes de fonctions supportent maintenant les types de retour structures :

`STRUCT IDENT IDENT '(' Parametres ')'`

Le premier IDENT est le nom du type structure, le second est le nom de la fonction.

De même, les paramètres peuvent être de type structure via la règle `ListTypVar`.

III – Construction de l'arbre abstrait

Nous utilisons systématiquement les fonctions fournies par tree.c :

makeNode() : création d'un nœud

addChild() : ajout d'un enfant

addSibling() : ajout d'un frère

siblingsToChildren() : conversion d'une liste chaînée de frères en enfants d'un même

parent

Cette dernière fonction est cruciale pour transformer les listes (déclarations, paramètres, instructions) en structure arborescente où tous les éléments sont enfants d'un même nœud

parent.

difficultés rencontrée

La plus grande difficulté rencontrée a été lors de l'implémentation initiale des structures. Notre grammaire présentait un conflit shift/reduce lié à l'ambiguïté entre :

- La déclaration d'un type structure : struct Point { ... };
- La déclaration de variables de type structure : struct Point p1, p2;

Lorsque le parser voyait STRUCT IDENT, il ne pouvait pas décider s'il devait s'attendre à une définition de structure (avec { }) ou à une liste de déclarateurs (noms de variables).

Solution : Nous avons fusionné les règles de déclarations globales (et locales) en une seule règle DeclGlobale (respectivement DeclLocale) qui accepte les trois formes :

1. Définition de structure
2. Déclaration de variable de type simple
3. Déclaration de variable de type structure

Cette fusion élimine l'ambiguïté car chaque alternative est clairement distincte dès le début de la règle.

Conclusion

Ce projet nous a permis de mettre en pratique les concepts d'analyse lexicale et syntaxique vus en cours. L'extension du langage TPC avec les structures nous a confrontés à des problématiques réelles de conception de grammaire et de résolution de conflits.

Les principales leçons tirées sont :

- L'importance de la conception de la grammaire pour éviter les ambiguïtés
- La nécessité de représenter correctement les structures de données dans l'arbre abstrait
- La gestion méthodique des cas particuliers (structures imbriquées, accès en chaîne)

Le résultat est un analyseur fonctionnel qui respecte les spécifications du sujet et qui peut servir de base pour la phase de compilation au second semestre.